Movie Recommender Application - Light Spec (REVISED)

General Description

Modern streaming services have made it easier than ever before to have thousands of options for entertainment available at user's fingertips. However, many users voice common frustrations with identifying and selecting a choice of film when there are ample options available. The Movie Recommender Application proposes a solution to the users decision paralysis quandary. The Movie Recommender Application would seek to help users decide what movie to watch based on several personalized selections. Users could search based on genre, release year, and rating instead of looking through 5,000+ movies per streaming service. The Movie Recommender application would focus on finding top-rated movies according to TMDB API (https://www.themoviedb.org/settings/api).

The API's functionality would allow users to make an API call to the TMDB movie database through the application. Users would then be able to pull the latest movie rating data based on their personal search preferences. The application version 1 could initially run as a simple command-line interface. Users could enter the type of movie they are interested in, along with filters such as release year and rating. For example, a user could search for action movies released after 2015 with a rating of at least 7.5. The program would use the TMDB API to find movies that match these criteria and display the highest-rated results to the user.

I would prefer to make the application web-based so that it can be accessed through a desktop web browser. For the user-facing end, I have been researching Streamlit and I believe it may be the best choice because it allows me to create a simple web interface using Python. The interface could include fields for selecting a movie genre, release year, and minimum rating, along with a button to search for recommendations. The results could then be displayed on the page using information retrieved from the TMDB API.

~~A more advanced version (final version) could recommend movies based on the user's previous preferences or ratings (stored during the session).~~ This project would demonstrate GUI design, searching, filtering, sorting, and data processing (see revised note in 7a).

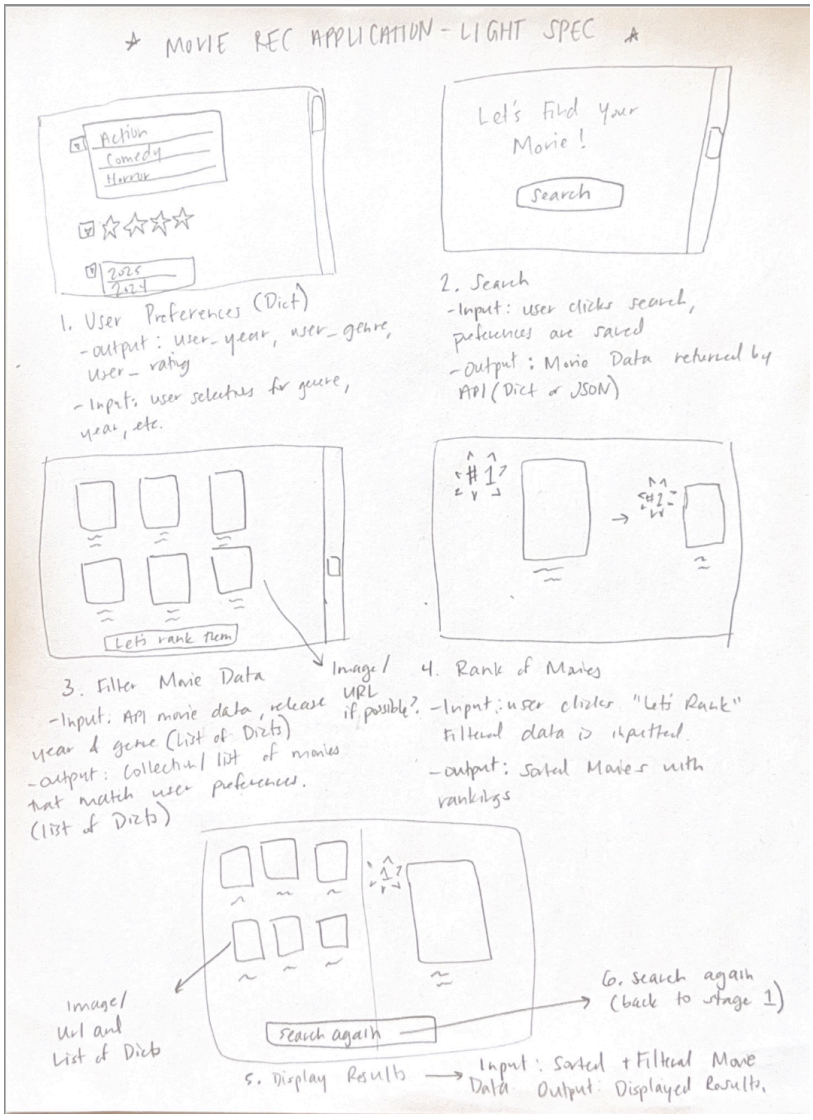
Task Vignettes/Overview

1. Enter preferences
    a. Alex wants to watch a movie but does not know what to choose. Alex selects the Action genre, chooses movies released from 2015 to the present, and sets the minimum rating to 7.5. Alex then submits these preferences.
    b. Technical:
        i. The app needs input options for genre, release date, and minimum rating.
        ii. A dropdown or selection box could be used for the genre.
        iii. A year range or two date/year selections could be used for the release dates.

    iv. User preferences need to be saved so that this information can be used for the search.
2. Search
    a. After Alex presses Next on their search, the application processes their selected preferences and displays a list of movies that match their criteria. Alex can then browse the results to find a movie they would like to watch.
    b. Technical:
        i. The application will send a request to the TMDB API.
        ii. The user's selected preferences will be included in the search request.
        iii. The application will handle cases where the search returns no movies.
3. Filter data
    a. The application filters the available movies based on Alex's selected preferences, including genre, release date, and minimum rating. Movies that do not meet Alex's criteria are removed from the results.
    b. Technical:
        i. The returned movie data will need to be examined and filtered.
        ii. The application will compare each movie's genre, release date, and rating with Alex's preferences.
        iii. Movies must meet all of the selected criteria to remain in the results.
        iv. Some information returned by the API may need to be converted into a format that Python can easily compare.
4. Rank movies
    a. The application ranks the movies that match Alex's preferences based on their ratings. Movies with higher ratings appear toward the top of the results. Movies with lower ratings are displayed on the lower end of the results.
    b. Technical:
        i. Python can sort the filtered movie data based on the movie rating.
        ii. The results should be sorted from highest rating to lowest rating.
        iii. The application could limit the number of results shown to Alex.
5. Display results
    a. Alex is shown a list of movies that match their preferences. Each movie displays basic information such as its title, rating, release date, and genre so Alex can decide which movie they may want to watch.
    b. (Nice to have) Technical:
        i. The application could also display movie posters using the poster information provided by TMDB - Will be implemented in VS2
        ii. Streamlit can be used to organize and display the movie results.
        iii. The application should provide a clear way for Alex to select a movie.
6. Select or change preferences
    a. Alex can select a movie from the results or change their search preferences if they are not satisfied with the recommendations. Alex can adjust options such as genre, release date, or minimum rating before searching again.
    b. Technical:
        i. The application needs a way to return to the preference selections.

   ~~ii. The previous preferences could remain selected so Alex does not have to start over…~~
   iii. Streamlit may be useful for keeping track of the user's selections.
   iv. Changing the preferences should start a new search using the updated criteria.
7. Search again
   a. After viewing the results, Alex can search again using different preferences ~~or select a movie to rate it (? maybe out of scope)~~. Alex's rating can be used to help personalize future movie recommendations.
      i. REVISED: I stepped back on my plan to have the ratings of the ranked movies stored (meaning after the application has selected a movie, the user would "rank" the movie themselves after watching). This is out of scope. The project seeks to provide users with recommendations, the central aim is not to store user ratings themselves.
   b. Technical:
      i. Alex should be able to start another search after rating a movie.

Technical Flow (see diagram attached)



Final Assessment:

1. After working through the spec as it's written, I think my biggest change is the main structure of the user flow. I believe I may have to step back on my plan to have ratings of the ranked movies stored (meaning after the application has selected a movie, the user would "rank" the movie themselves after watching). I think this is too out of scope, and right now doesn't really pertain to the main function of the application. Instead, I think I will stick with the "search again" user flow.
    a. REVISED: I stepped back on my plan to have the ratings of the ranked movies stored (meaning after the application has selected a movie, the user would "rank" the movie themselves after watching).
2. I feel fairly confident. I still need to do a bit of work to understand how to connect the TMBD API and get the two systems (my python and the API) to talk to each other. I think the GUI creation will also be something I need to review/refresh my memory on.

However, I feel with my outline drawn above, I have a good starting point of what I see for version 1.

3. The biggest potential problem I may encounter is when a user's preferences do not match any movie criteria… Do I recommend they restart or do I match them with the next closest option? I am unsure.. I will need to re-work this into user flows later.
    a. REVISED: As written, the VS 1 code provides users with the next closest option. I believe this achieves the true aim of users - to find the closest match to their criteria.
4. As I mentioned above, I think the connection between the API, maybe Streamlit, and also Python is something I will likely need help with. I am excited to learn more about it though!